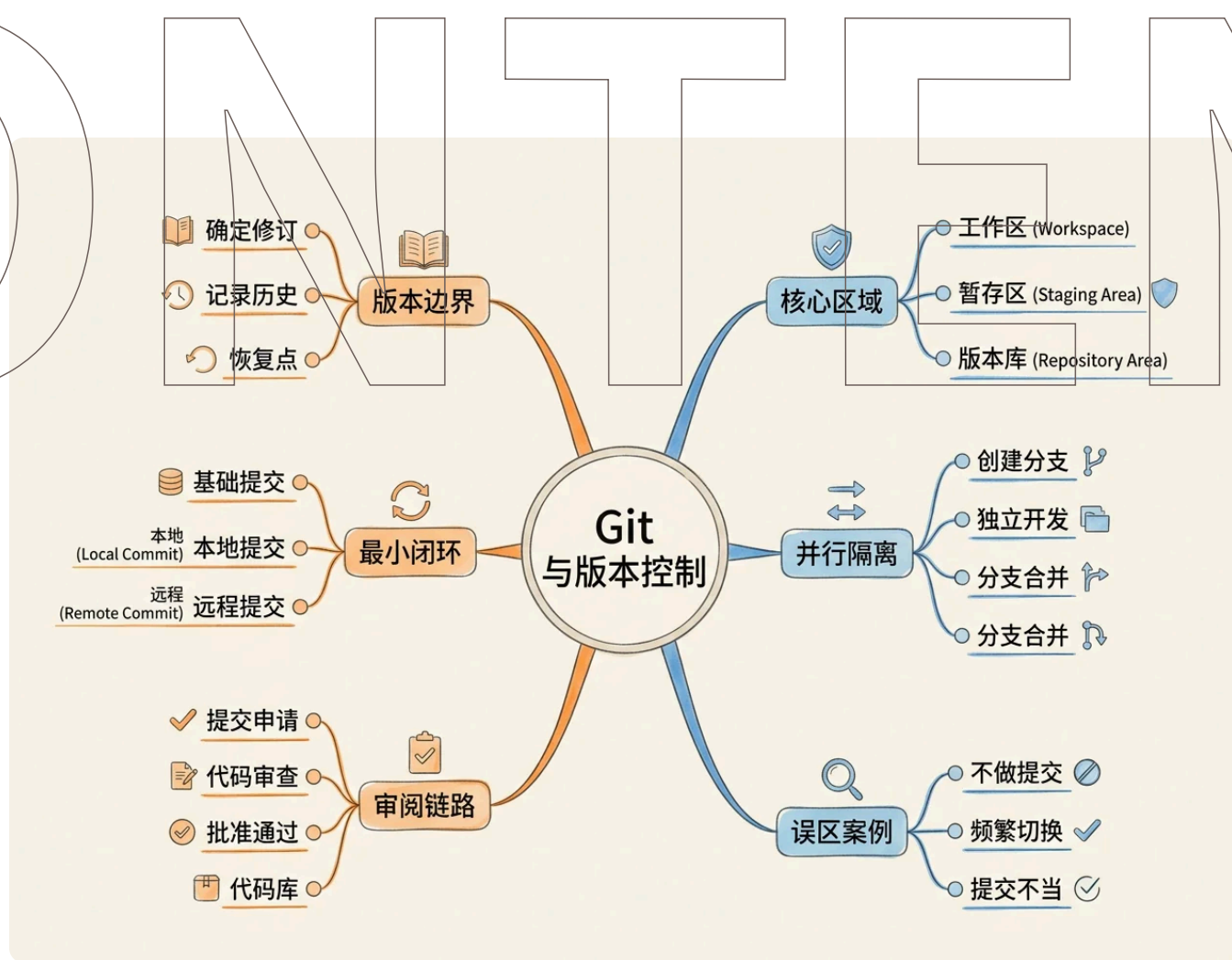


第 6 章 Git 与版本控制

张学人

安徽大学经济学院



全章六节按操作链展开。先说明为什么项目创建后要立刻接上 Git，再讲工作区、暂存区和提交点三层结构，然后走一遍从 `status` 到 `commit` 的本地提交流程。接着介绍分支、Worktree 和并行任务隔离，之后收束常见误区，最后用两个经济金融案例串联全章命令。

6.1 AI中的Git简介

6.1.1 为什么AI协作项目需要 Git

传统手写流程中，一次只改一两个地方

- 能靠记忆维持修改先后顺序
- word中的修订功能
- 或者手动的版本控制：第N版/最后稿/最后终稿

AI 介入后，容易触发以下问题：

- 修改速度明显加快，个人审阅呈现出一波一波的情况，应接不暇，容易忽略出错
- 模型修改内容晦涩不明，会出现暂时无法判断正误的情形

如果不加以控制，可能会有更加糟糕的情况出现：

1. diff 太大，只能整包粗看，人工审阅失效。
2. 出错时没有明确落点，只能靠聊天记录追溯。

6.1.2 Git 的历史

版本控制系统（Version Control System）的工作，是把文件的每一次修改都记录下来，让你可以查看历史、比较差异，并在需要时回退到任意一个过去的版本。它用于防止只靠文件名和记忆来维系造成的丢失、混乱、出错。

Git 诞生于 **2005 年**。在此之前，Linux 内核项目使用一个叫 BitKeeper 的商业版本控制工具。当 BitKeeper 取消了对开源项目的免费授权后，Linus Torvalds（Linux 操作系统的创始人）用大约两周时间写出了 Git 的初版，专门用于管理 Linux 内核这一庞大的协作项目。

设计目标	含义
速度	Linux 内核改动量极大，工具必须快
去中心化	每个副本都是完整仓库，不依赖中央服务器，断网也能工作
数据完整性	每次提交都有唯一的哈希标识，历史无法被篡改

这三个目标决定了 Git 和当时主流工具（如 SVN）的根本差异。SVN 需要一台中央服务器，离线时无法提交；Git 的每个副本都是完整仓库，离线也能正常工作。**2008 年 GitHub 上线**，在 Git 之上增加了可视化界面、Pull Request（合并请求）和 Actions（自动化流水线）等协作功能，使 Git 从程序员的命令行工具演变为软件行业的标准基础设施。

6.1.3 Git在AI协作中的作用

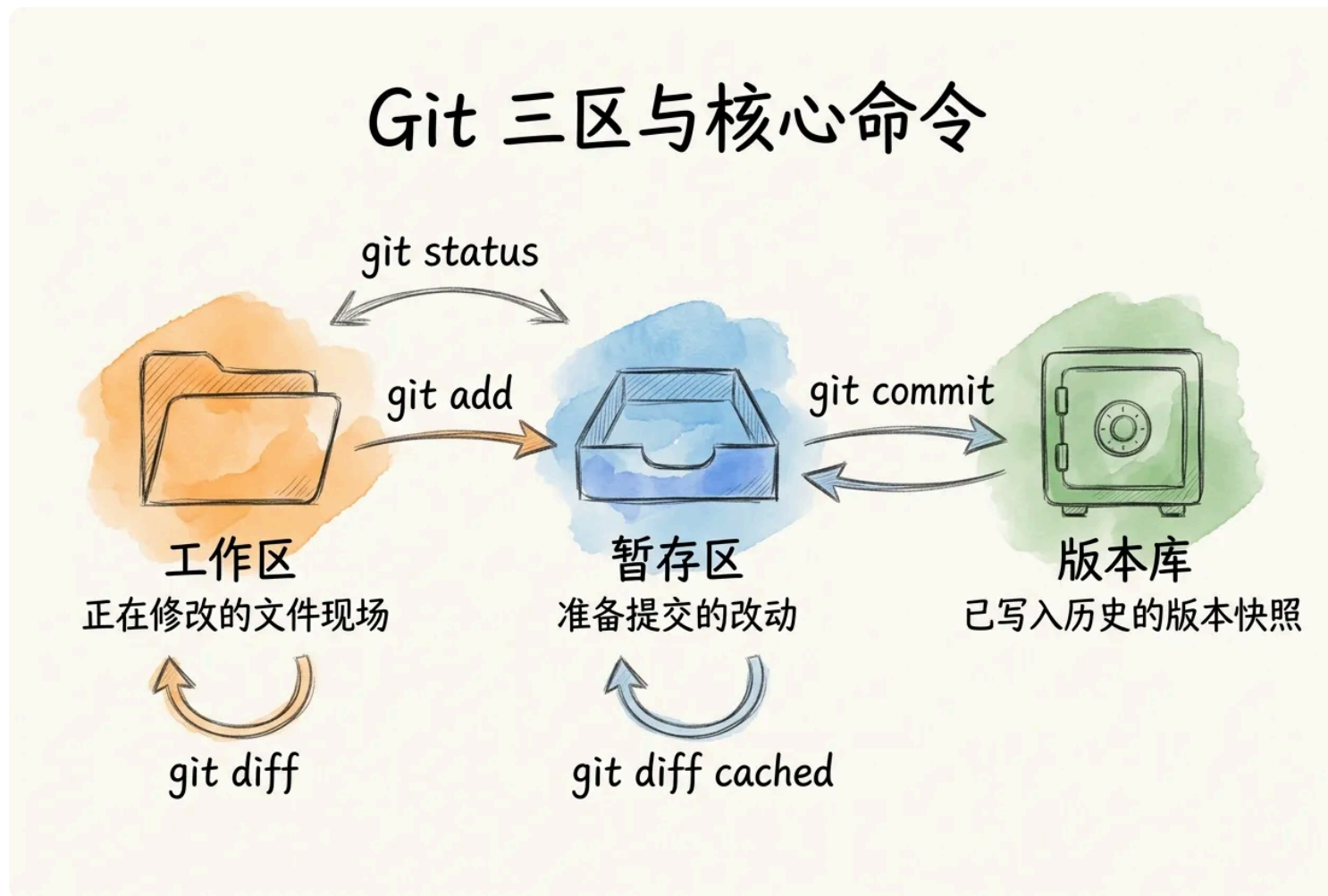
项目目录解决文件秩序，任务说明书解决规划秩序，而 Git 解决的是变更秩序，明确每一次变更的内容，使得后续的 Skill、子代理、Hooks 和评估能够有所参照。

在 AI 协作里，Git 至少承担四个角色：

角色	作用
记录	把改动从聊天过程转成可检查的版本节点
审阅	让人围绕 diff 和提交点做判断
回退	让试错可以收住，而不是越补越乱
交接	让后续会话、队友或未来的自己知道这一轮做过什么

6.2 本地提交流程

6.2.1 工作区、暂存区、提交点



很多读者第一次学 Git，面对大量术语容易感到困惑。但初次了解只要先把三个位置分清：正在修改的地方、准备提交的地方、已经进入历史的地方。

6.2.1 工作区、暂存区和提交点

- 工作区：是当前文件的实际状态，你和AI此刻正在改的文件现场
- 暂存区：是下次提交的预备区域，你准备放进下一次提交的那部分改动
- 提交点：是已写入历史的版本快照，可以比较和回退的版本节点

对应的最常用命令如下：

命令	作用
git status	看现场状态
git diff	看工作区相对上一次提交改了什么
git add <file>	把指定改动放进暂存区
git diff --cached	看暂存区里准备提交什么
git commit -m "..."	形成新的提交点

6.2.2 初始化

如果项目还没有 Git 仓库，先在项目根目录执行：

```
git init  
git status
```

- `git init` 会在当前目录创建 Git 仓库。
- `git status` 用来确认现场：哪些文件尚未跟踪，哪些文件已经修改，当前是否已经有提交历史。

6.2.3 查看更改

最容易出现的问题，是让智能体改完以后直接提交。更稳妥的顺序应该是先看改动，再暂存：

```
git status  
git diff
```

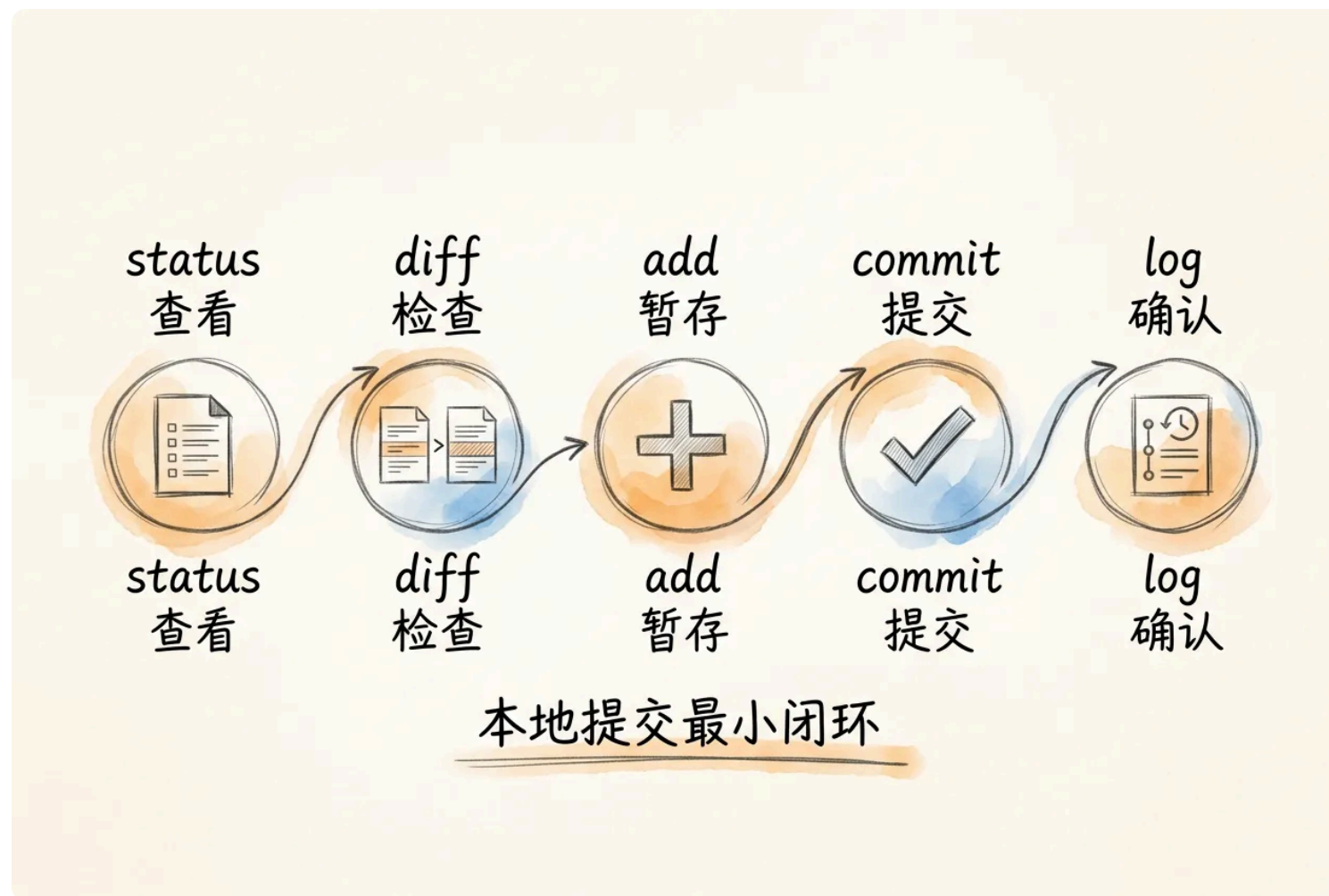
如果只想看某个文件，命令更具体：

```
git diff chapters/chapter-6/section-6-3.qmd
```

如果你已经执行过 `git add`，可以通过 `--cached` 参数查看暂存区的修订

```
git diff --cached
```

6.2.4 本地提交流程：从 status 到 commit



6.2.4 本地提交流程：从 `status` 到 `commit`

假设你刚让智能体改完一轮章节内容，现在准备把这一轮修订提交：

```
git status
git diff
git add chapters/chapter-6/section-6-1.qmd
git add chapters/chapter-6/section-6-2.qmd
git diff --cached
git commit -m "docs: rewrite git chapter for hands-on workflow"
```

1. `git status` 看改动范围。
2. `git diff` 检查正文到底改了什么。
3. `git add` 只挑本次要提交的文件。
4. `git diff --cached` 再确认一次暂存区内容。
5. `git commit` 形成提交点。

如果改动范围很大，不要直接 `git add .`。这会把附带修改、临时文件和无关调整一并提交。

补充1:提交信息 `git commit -m "sometext"`

提交信息不需要复杂，但要让人看得出这次提交的目标。比较稳妥的写法是：

类型：这次提交解决了什么

例如：

```
docs: reorder part-two chapters and rewrite git workflow  
docs: tighten preface wording for new chapter order  
chore: add quarto selection mapping for chapter 6
```

一条提交只说一件事，后续回顾/回退时看得更方便。

“ 每次提交只包含一个明确目的的改动。提交越小，回退越精准，审阅越容易，出问题时排查范围也越窄。与 AI 协作时尤其如此——模型单轮改动范围往往较大，拆成多个小提交能有效控制风险。

补充2:提交后确认

本地提交完成后，你还可以做以下两步：

```
git log --oneline -n 5  
git show --stat --oneline HEAD
```

第一条确认提交已经进入历史。第二条确认这一版到底触及了哪些文件、规模是否合理。

补充3：推送到远程仓库

本地提交只存在于你的机器上。如果需要备份、共享给队友，或者触发后续的代码审阅和自动化流程，就要把本地历史推送到远程仓库（如 GitHub）。

第一次：关联远程仓库地址

```
git remote add origin https://github.com/yourname/yourrepo.git
```

把当前分支推送到远端，并建立跟踪关系

```
git push -u origin main
```

之后每次推送，只需要

```
git push
```

推送之后，队友可以用 `git pull` 把你的最新提交拉取到本地；你也可以在 GitHub 上发起 **Pull Request**，让其他人围绕这批提交做评审，再决定是否合入主线。

“ 本地提交解决的是“有没有历史”的问题，推送解决的是“历史能否共享和审阅”的问题。AI 协作里先把本地提交习惯建立起来，再接上推送和 PR， workflow 就完整了。

6.3 回退、分支与worktree

6.3.1 本地回退

初学阶段最常用的本地回退主要有三类：

丢弃工作区中尚未暂存的修改

```
git restore chapters/chapter-6/section-6-3.qmd
```

取消暂存，但保留文件修改

```
git restore --staged chapters/chapter-6/section-6-3.qmd
```

回看某个历史提交

```
git show HEAD~1
```

6.3.2 分支

只要任务满足下面任意一条，就值得创建分支：

- 这轮修改会持续一段时间，不适合直接压在主线上
- 需要单独审阅
- 失败时要局部回退
- 同时还有另一条改动线在推进

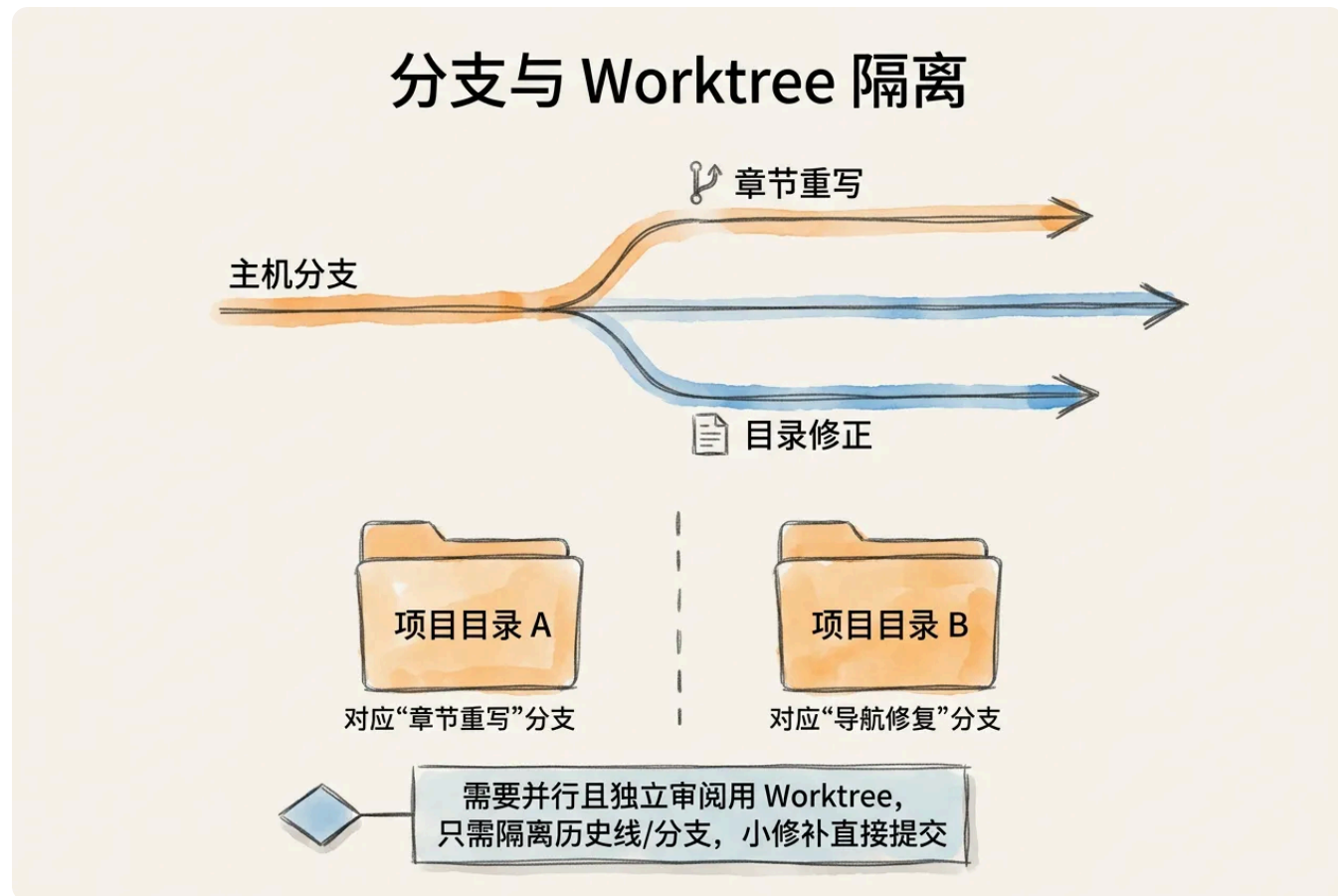
最常用的创建方式：

```
git switch -c chapter6-git-rewrite
```

这条命令会新建并切换到 `chapter6-git-rewrite` 分支。之后这条分支上的提交，就只服务这一轮目标。

创建分支的好处是改动被隔离在独立分支上，主线代码始终可用。试验成功就合入，失败就丢弃，不需要手动撤销每一处修改。

6.3.3 worktree



6.3.3 Worktree 的作用

分支隔离的是历史线，Worktree 隔离的是工作目录。对 AI 协作来说，目录隔离很关键，因为一个会话通常默认围绕当前目录展开。

手动方式：

```
git worktree add ../eco-ai-agent-book-design-navfix nav-fix
```

这条命令会在相邻目录创建一个新工作树，并把 `nav-fix` 分支检出到那里。这样，你可以让另一条会话在另一个目录里单独推进导航修复，不会和当前目录里的章节重写混在一起。

6.3.3 Worktree 的判断标准

如果任务只是很小的局部修补，而且不会和其他任务并行，直接在当前分支上小步提交就够了。只有当下面两条同时成立时，才优先考虑 Worktree：

- 1. 任务要并行推进。
- 2. 每条任务线都需要独立审阅和独立回退。

“ 例如同一天并行处理三件事：重写第 6 章 Git 正文;修第 11 章 Hooks 目录映射;调整前言中的第二篇顺序说明

更稳妥的方式是：

任务	建议隔离方式
Git 正文重写	一个独立分支
Hooks 目录修正	另一个分支，必要时单独 Worktree
前言顺序调整	若改动很小，可并入同一分支；若要反复试改，单独分支更稳妥

6.4 本章核心概念速查

概念	定义	何时使用	常见错误
git init	初始化本地仓库	新项目创建后立刻执行	项目做到一半才初始化
git add	将改动加入暂存区	选定本次要提交的文件时	用 git add . 全部暂存
git commit	把暂存区内容提交为版本节点	完成一个小目标后	攒太多改动一次提交
git diff	查看工作区与暂存区的差异	提交前确认改动范围时	不看 diff 直接提交
git log	查看提交历史列表	确认提交是否成功、回溯改动顺序时	只靠聊天记录回忆改了什么
git push	把本地提交推送到远程仓库	需要备份、共享或触发 PR 审阅时	只在本地提交，从不推送
Worktree	同一仓库的多个独立工作目录	需要并行处理多个任务时	和分支混为一谈
小步提交	每次只提交一个单一目标的改动	AI 协作中控制变更边界时	把多个目标混在一次提交

要点小结

第一，Git 应该接在项目创建和任务规划之后。

因为目录和规则文件只解决现场秩序，Git 解决的是改动秩序。

第二，本地提交是最关键的起点。

先把 `status -> diff -> add -> diff --cached -> commit -> log` 这条流程链掌握，PR、CI 和 Hooks 才有稳定入口。

第三，分支和 Worktree 解决的是隔离问题。

分支隔离历史线，Worktree 隔离工作目录。并行任务进入工程化状态时，两者都很重要。

第四，自动化不能替代人工判断。

Hooks 和 CI 可以筛查问题，但这条改动是否应该进入主线仍需要明确的人类判断位。

最值得记住的一句话是：**AI 让修改更快，Git 让修改可控。**